

HB using XML

Simple example of Hibernate Application

Steps:

1. Create the maven project

Create the java project by **File Menu - New - project – Maven Project**. Provide the appropriate information(s).

2. Create the package

Right click on **src/main/java** and select **New – Package** and create a package **com.company.mypackage**.

3. Create the Persistent class

To create the persistent class, Right click on **com.company.mypackage** - **New - Class** - specify the class (eg. Employee.java) with package name (e.g. com.company.mypackage) - **finish**.

Employee.java

```
package com.company.mypackage;
```

```
public class Employee {  
    private int id;  
    private String firstName,lastName;  
    public int getId() {  
        return id;  
    }  
    public String getFirstName() {  
        return firstName;  
    }  
    public String getLastName() {  
        return lastName;  
    }  
    public void setId(int id) {  
        this.id = id;  
    }  
    public void setFirstName(String firstName) {  
        this.firstName = firstName;  
    }  
    public void setLastName(String lastName) {  
        this.lastName = lastName;  
    }  
}
```

}

4. Create the mapping file for Persistent class

To create the mapping file, Right click on **src/main/java** - **new** - **file** - specify the file name (e.g. employee.hbm.xml) - **ok**. It must be outside the package.

employee.hbm.xml

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 5.3//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
  <class name="com.company.mypackage.Employee" table="emp1000">
    <id name="id">
      <generator class="assigned"></generator>
    </id>
    <property name="firstName"></property>
    <property name="lastName"></property>
  </class>
</hibernate-mapping>
```

5. Create the Configuration file

The configuration file contains all the information for the database such as connection_url, driver_class, username, password etc. The hbm2ddl.auto property is used to create the table in the database automatically. To create the configuration file, right click on src - new - file. Now specify the configuration file name e.g. hibernate.cfg.xml.

hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property
name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriv
er</property>
    <property
name="hibernate.connection.password">oracle</property>
```

```

        <property
name="hibernate.connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
roperty>
        <property
name="hibernate.connection.username">system</property>
        <property
name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</prope
rty>
        <property name="hbm2ddl.auto">update</property>
        <property name="show_sql">true</property>
        <mapping resource="employee.hbm.xml"/>
    </session-factory>
</hibernate-configuration>

```

6. Create the class that retrieves or stores the persistent object

In this class, we are simply storing the employee object to the database.

```

package com.company.mypackage;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;

public class StoreData {
    static Session getSession() {
        Configuration config = new Configuration()
            .configure("hibernate.cfg.xml");
        ServiceRegistry sr = new ServiceRegistryBuilder()
            .applySettings(config.getProperties())
            .buildServiceRegistry();
        SessionFactory factory = config.buildSessionFactory(sr);
        Session session = factory.openSession();
        return session;
    }

    public static void main( String[] args )
    {
        Session session = getSession();
        Transaction t = session.beginTransaction();
        Employee e1=new Employee();
        e1.setId(1);
    }
}

```

```

        e1.setFirstName("Rohan");
        e1.setLastName("kumar");
        session.save(e1);
        t.commit();
        System.out.println("Successfully saved");
        session.close();
    }
}

```

7. Edit the pom.xml file

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>MavenProject</groupId>
    <artifactId>com.java</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <dependencies>
        <!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-
core -->
        <dependency>
            <groupId>org.hibernate</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>4.1.6.Final</version>
        </dependency>
        <!--
https://mvnrepository.com/artifact/com.oracle.database.jdbc/ojdbc6 -
->
        <dependency>
            <groupId>com.oracle.database.jdbc</groupId>
            <artifactId>ojdbc6</artifactId>
            <version>11.2.0.4</version>
        </dependency>
    </dependencies>
</project>

```

8. Run the application

To run the hibernate application; right click on the StoreData class - Run As - Java Application.